Sri Sivasubramaniya Nadar College of Engineering,
Kalavakkam - 603 110
(An Autonomous Institution, Affiliated to Anna University, Chennai)

Course Code	UIT2702	Course Name	Cloud and Distributed Computing
Regulation	2021	Academic Year	2026-2027
Semester		VII	
Department Offering the Course		Information Technology	
Exercise: 04		Clock and Time synchronization	

Name: Rahul Akshath Rajendran

Reg No: 3122235002095

Sem / Sec: VII B

Exp. No: 04 **Date: 29-07-2026**

CLOCK AND TIME SYNCHRONIZATION

AIM

To implement logical clocks and vector clocks in order to manage the ordering of events and to capture the causality relationship between events in a distributed system.

QUESTION

Objective: To implement logical clocks and vector clocks to manage event ordering and causality in distributed systems.

Task 1 - Logical Clocks (Lamport's Logical Clock):

- Initialize each process with a logical clock.
- For each event or message sent, increment the clock and send the timestamp.
- On message receipt, update the clock based on the received timestamp, ensuring ordered events.

Task 2 - Vector Clocks:

- Set up each process with a vector representing the state of all processes.
- For each event or message sent, update the process's vector and transmit it.
- Upon receipt, merge the vectors to reflect concurrent events and causality relationships.

ALGORITHM / STEPS

Task 1 - Lamport's Logical Clock

1. Each process Pi keeps a single integer logical clock, initialized to 0, and listens for messages on the port 5000 + i.

2. A background listener thread waits for messages coming from the other processes.

3. Internal event: whenever the process performs a local (internal) event it increments its own clock by one, that is clock = clock + 1.

4. Send event: before sending a message the process increments its clock by one and attaches this value as the timestamp of the message, then transmits (sender id, timestamp) to the destination process.

5. Receive event: when a message with timestamp ts arrives, the process updates its clock using Lamport's rule clock = max(clock, ts) + 1. This guarantees that the receive event always has a larger timestamp than the matching send event.

6. Because the clock never decreases and every message increases the receiver's clock beyond the sender's, the happened-before relation (a -> b implies C(a) < C(b)) is preserved, which gives a consistent ordering of events across the whole system.

Task 2 - Vector Clock

1. Each process Pi keeps a vector of N counters (one per process), all initialized to 0, and listens on the port 5100 + i. The entry vector[i-1] counts the events of this process and the other entries record how much this process knows about the others.
2. Internal event: the process increments only its own position, that is vector[ME-1] = vector[ME-1] + 1.
3. Send event: the process first increments its own position and then sends the entire vector along with its id to the destination process.
4. Receive event: when a vector is received, the process merges it element by element using vector[k] = max(vector[k], received[k]) for every k, and then increments its own position vector[ME-1] = vector[ME-1] + 1 to count the receive event.
5. Comparing two vectors tells the exact relationship between two events: if every element of one vector is less than or equal to the other then the events are causally ordered; if neither vector dominates the other then the two events are concurrent.
6. In this way the vector clock captures both causality and concurrency, which a single Lamport clock cannot distinguish.

CODE

Task 1 - Lamport's Logical Clock

lamport.py

```
 import socket
import threading
import sys

ME = int(sys.argv[1])
N = 3
PORT = 5000 + ME

clock = 0
```

```
def send(pid, msg):
    try:
        s = socket.socket()
        s.connect(("localhost", 5000 + pid))
        s.send(msg.encode())
        s.close()
    except:
        pass

def listener():
    global clock
    server = socket.socket()
    server.bind(("localhost", PORT))
    server.listen(5)
    server.settimeout(1)
    try:
        while True:
            try:
                conn, addr = server.accept()
                sender, ts = conn.recv(1024).decode().split()
                conn.close()
                clock = max(clock, int(ts)) + 1
                print("Received from P%s (ts=%s) -> clock = %d" % (sender, ts, clock))
            except socket.timeout:
                pass
    finally:
        server.close()

threading.Thread(target=listener, daemon=True).start()

print("Process", ME, "started (Lamport clock)")
print("Commands:  e = internal event   s = send message   q = quit")

try:
    while True:
        cmd = input()

        if cmd == "e":
```

```
            clock += 1
            print("Internal event -> clock = %d" % clock)

        elif cmd == "s":
            pid = int(input("Send to process (1/2/3): "))
            clock += 1
            send(pid, "%d %d" % (ME, clock))
            print("Sent to P%d -> clock = %d" % (pid, clock))

        elif cmd == "q":
            break

except KeyboardInterrupt:
    Pass
```

Execution: open three terminals and run "python3 lamport.py 1", "python3 lamport.py 2" and "python3 lamport.py 3". In any terminal type 'e' for an internal event, 's' to send a message (then enter the destination process number), or 'q' to quit.

Task 2 - Vector Clock

vector.py

```
import socket
import threading
import sys

ME = int(sys.argv[1])
N = 3
PORT = 5100 + ME

vector = [0] * N

def send(pid, msg):
    try:
        s = socket.socket()
```

```
        s.connect(("localhost", 5100 + pid))
        s.send(msg.encode())
        s.close()
    except:
        pass

def listener():
    global vector
    server = socket.socket()
    server.bind(("localhost", PORT))
    server.listen(5)
    server.settimeout(1)
    try:
        while True:
            try:
                conn, addr = server.accept()
                sender, values = conn.recv(1024).decode().split("|")
                conn.close()
                received = list(map(int, values.split()))
                for i in range(N):
                    vector[i] = max(vector[i], received[i])   # merge vectors
                vector[ME - 1] += 1                          # receive event
                print("Received from P%s %s -> vector = %s" % (sender, received,
vector))
            except socket.timeout:
                pass
    finally:
        server.close()

threading.Thread(target=listener, daemon=True).start()

print("Process", ME, "started (Vector clock)")
print("Commands:  e = internal event   s = send message   q = quit")

try:
    while True:
        cmd = input()
```

```
        if cmd == "e":
            vector[ME - 1] += 1
            print("Internal event -> vector = %s" % vector)

        elif cmd == "s":
            pid = int(input("Send to process (1/2/3): "))
            vector[ME - 1] += 1
            send(pid, "%d|%s" % (ME, " ".join(map(str, vector))))
            print("Sent to P%d -> vector = %s" % (pid, vector))

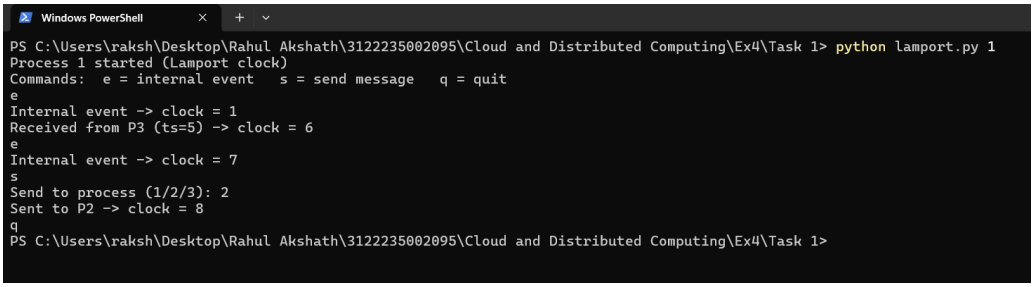
        elif cmd == "q":
            break

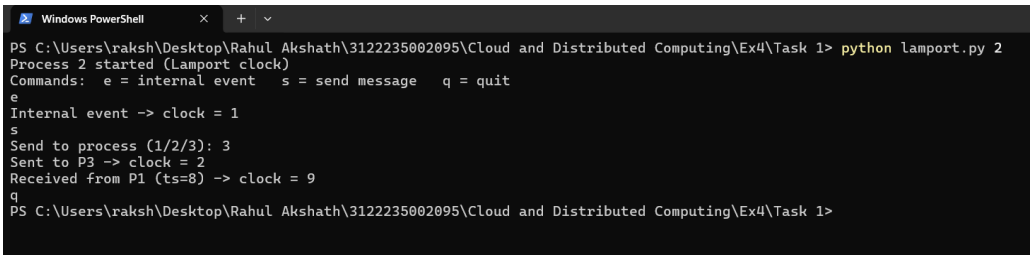
except KeyboardInterrupt:
    Pass
```

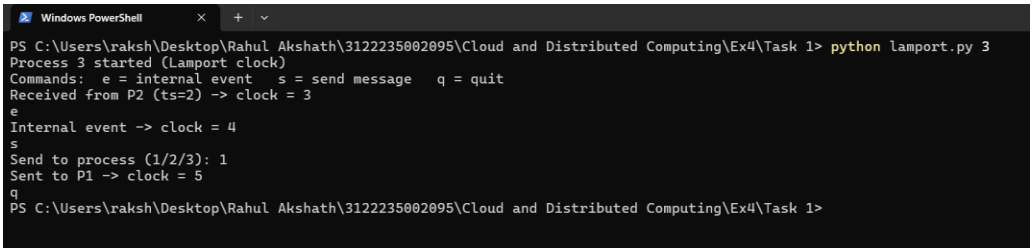
Execution: open three terminals and run "python3 vector.py 1", "python3 vector.py 2" and "python3 vector.py 3". The commands 'e', 's' and 'q' work exactly as in Task 1.

OUTPUT SCREENSHOTS

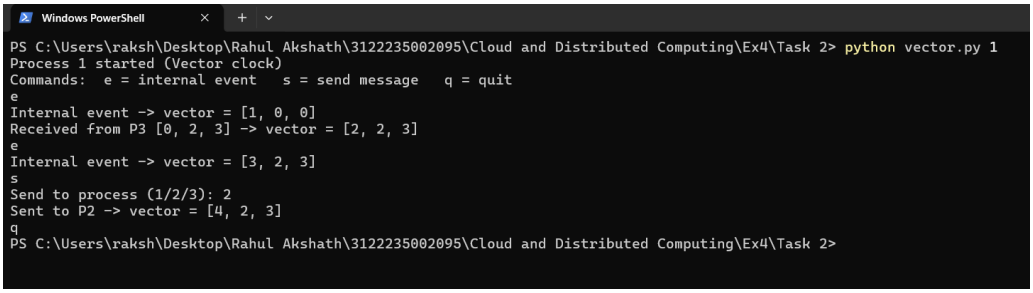
Task 1 - Lamport's Logical Clock

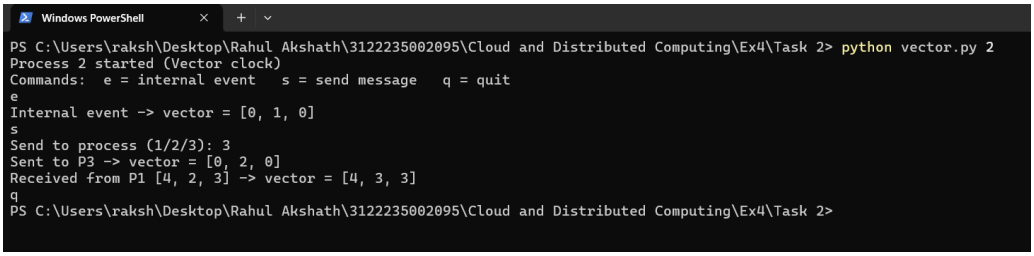


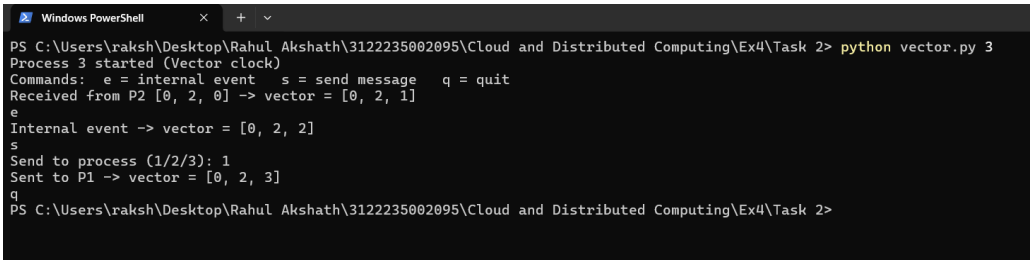




Task 2 - Vector Clock







CONCLUSION

Lamport's logical clock and the vector clock were implemented successfully using TCP sockets, with each process running its own clock and communicating only through messages. In Task 1 the Lamport clock assigned a single increasing integer timestamp to every event and, by applying the rule clock = max(clock, ts) + 1 on receipt, it preserved the happened-before ordering of causally related events; however, it cannot tell whether two events are concurrent. In Task 2 the

vector clock overcame this limitation: by keeping one counter per process and merging vectors on message receipt, it was able to distinguish causally ordered events from concurrent ones by simply comparing the vectors. The observed output matched the expected values in both cases, so the ordering of events and the detection of causality and concurrency in a distributed system were understood and verified.